



Name: Rahin Ibne Harun

Id: 23-54177-3

Sec: U

Course: Compiler Design Assignment

Lab Task-5

Task Question:

You are given an input file named **input.txt**. Each line contains a simple C/C++ like statement.

Step A, Token classification

Read each line and identify tokens from the following categories:

1. Keyword

Use only these keywords:

int, float, double, char, void, if, else, while, return

2. Identifier

A valid identifier:

- starts with a letter or underscore
- remaining characters may be letters, digits, or underscore

3. Integer Constant

A token containing only digits

4. Operator

Use these operators only:

+ - * / = < >

5. Delimiter

Use these delimiters only:

(){}[] , ;

Ignore spaces while reading.

Step B, Write classified tokens to a file

Create a file named **tokens.txt**.

For each token, write its class in this format:

Line <n>: <token> -> <class>

Step C, Build a simple symbol table

Create another file named **symbol_table.txt** containing only the **unique identifiers** found in the input.

Write them in this format:

<serial> <identifier> <first line number>

If the same identifier appears again later, do not insert it again.

Step D, Add summary

At the end of **tokens.txt**, add the total count of:

- Keywords
- Identifiers
- Integer Constants
- Operators
- Delimiters

Answer:

```
#include <iostream>
```

```
#include <fstream>
```

```
#include <string>
```

```
using namespace std;
```

```
string tokenValues[500];
```

```
string tokenTypes[500];
```

```
int tokenLines[500];
```

```
int tokenCount = 0;
```

```
string identifiers[100];
```

```
int firstLine[100];
```

```
int idCount = 0;
```

```
int keywordCount = 0;
```

```
int identifierCount = 0;
```

```
int constantCount = 0;
```

```
int operatorCount = 0;
```

```
int delimiterCount = 0;
```

```
bool isKeyword(string word) {
```

```
    if(word == "int" || word == "float" || word == "double" || word == "char" || word == "void" || word == "if"  
    || word == "else" || word == "while" || word == "return") {
```

```
        return true;
```

```
    }
```

```
    return false;

}
```

```
bool isOperator(char ch) {

    if(ch == '+' || ch == '-' || ch == '*' || ch == '/' || ch == '=' || ch == '<' || ch == '>') {

        return true;

    }

    return false;

}
```

```
bool isDelimiter(char ch) {

    if(ch == '(' || ch == ')' || ch == '{' || ch == '}' || ch == '[' || ch == ']' || ch == ';' || ch == ',') {

        return true;

    }

    return false;

}
```

```
bool isDigit(char ch) {

    if(ch >= '0' && ch <= '9') return true;

    return false;

}
```

```
bool isLetter(char ch) {
```

```
if((ch >= 'a' && ch <= 'z') || (ch >= 'A' && ch <= 'Z') || ch == '_') {  
  
    return true;  
  
}  
  
return false;  
  
}
```

```
bool isIntegerConstant(string token) {  
  
    for(int i = 0; i < token.length(); i++) {  
  
        if(!isDigit(token[i])) return false;  
  
    }  
  
    return true;  
  
}
```

```
bool isValidIdentifier(string token) {  
  
    if(token.length() == 0) return false;  
  
    if(!isLetter(token[0])) return false;  
  
  
    for(int i = 1; i < token.length(); i++) {  
  
        if(!isLetter(token[i]) && !isDigit(token[i])) {  
  
            return false;  
  
        }  
  
    }  
  
    return true;  
  
}
```

```
void addToken(string value, string type, int line) {  
  
    tokenValues[tokenCount] = value;  
  
    tokenTypes[tokenCount] = type;  
  
    tokenLines[tokenCount] = line;  
  
    tokenCount++;  
  
}
```

```
void addToSymbolTable(string identifier, int line) {  
  
    for(int i = 0; i < idCount; i++) {  
  
        if(identifiers[i] == identifier) {  
  
            return;  
  
        }  
  
    }  
  
    identifiers[idCount] = identifier;  
  
    firstLine[idCount] = line;  
  
    idCount++;  
  
}
```

```
void processToken(string token, int lineNo) {  
  
    if(isKeyword(token)) {  
  
        addToken(token, "Keyword", lineNo);  
  
        keywordCount++;  
  
    }  
  
    else if(isIntegerConstant(token)) {  
  
        addToken(token, "Integer Constant", lineNo);  
  
    }  
  
}
```

```

        constantCount++;

    }

    else if(isValidIdentifier(token)) {

        addToken(token, "Identifier", lineNo);

        identifierCount++;

        addToSymbolTable(token, lineNo);

    }

}

int main() {

    ifstream inputFile("input.txt");

    if(!inputFile.is_open()) {

        cout << "Error: Cannot open input.txt" << endl;

        return 1;

    }

    string line;

    int lineNo = 0;

    while(getline(inputFile, line)) {

        lineNo++;

    }

    string currentToken = "";

    for(int i = 0; i < line.length(); i++) {

        char ch = line[i];

```

```
if(ch == ' '){

    if(currentToken != "") {

        processToken(currentToken, lineNo);

        currentToken = "";

    }

    continue;

}

if(isOperator(ch)) {

    if(currentToken != "") {

        processToken(currentToken, lineNo);

        currentToken = "";

    }

    string op(1, ch);

    addToken(op, "Operator", lineNo);

    operatorCount++;

    continue;

}
```

```
if(isDelimiter(ch)) {

    if(currentToken != "") {

        processToken(currentToken, lineNo);

        currentToken = "";

    }

}
```



```
string delim(1, ch);
```

```
addToken(delim, "Delimiter", lineNo);
```

```
delimiterCount++;
```

```
continue;
```

```
}
```

```
currentToken += ch;
```

```
}
```

```
if(currentToken != "") {
```

```
processToken(currentToken, lineNo);
```

```
}
```

```
}
```

```
inputFile.close();
```

```
ofstream tokenFile("tokens.txt");
```

```
for(int i = 0; i < tokenCount; i++) {
```

```
    tokenFile << "Line " << tokenLines[i] << ": " << tokenValues[i] << " -> " << tokenTypes[i] << endl;
```

```
}
```

```
tokenFile << "\nSummary:\n";
```

```
tokenFile << "Keywords = " << keywordCount << endl;
```

```
tokenFile << "Identifiers = " << identifierCount << endl;
```

```
tokenFile << "Integer Constants = " << constantCount << endl;

tokenFile << "Operators = " << operatorCount << endl;

tokenFile << "Delimiters = " << delimiterCount << endl;

tokenFile.close();

ofstream symbolFile("symbol_table.txt");

for(int i = 0; i < idCount; i++) {

    symbolFile << (i+1) << " " << identifiers[i] << " " << firstLine[i] << endl;

}

symbolFile.close();

cout << "Done! tokens.txt and symbol_table.txt created successfully." << endl;

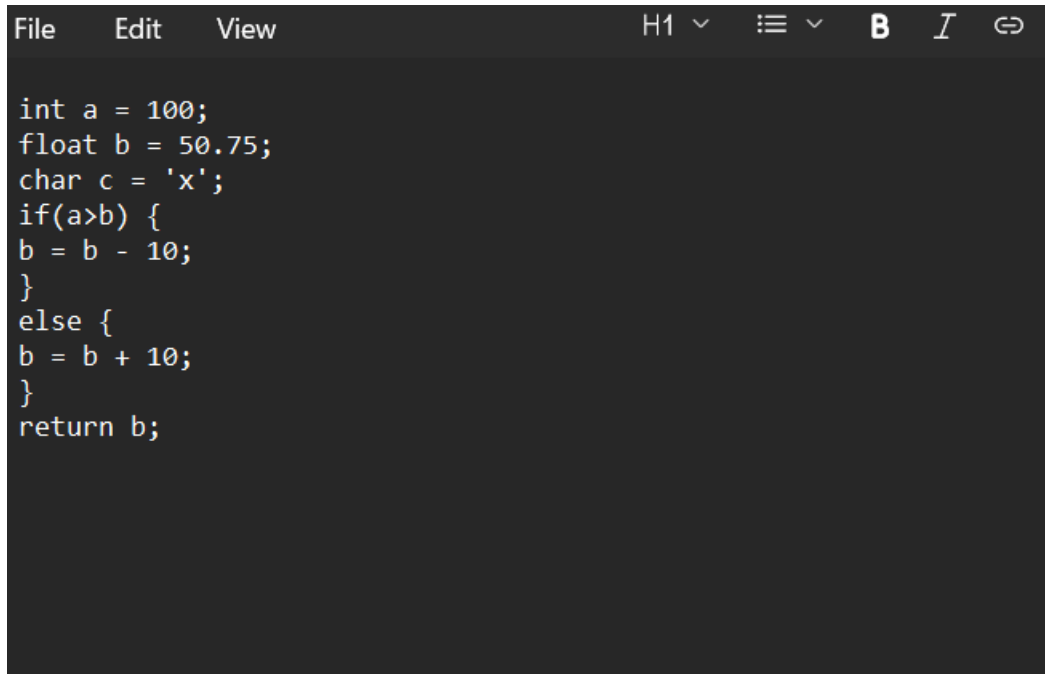
cout << "Total tokens founding number : " << tokenCount << endl;

cout << "Total identifiers number: " << idCount << endl;


return 0;

}
```

Input file ss:

A screenshot of a code editor window with a dark theme. The window has a menu bar with 'File', 'Edit', and 'View'. On the right side of the menu bar, there are icons for font size (H1 with a dropdown arrow), a list icon (three horizontal lines with a dropdown arrow), bold (B), italic (I), and a link icon (chain link). The code is written in a light gray font on a dark background. It defines an integer 'a' as 100 and a float 'b' as 50.75. A character 'c' is defined as 'x'. An if statement checks if 'a' is greater than 'b'. If true, 'b' is decreased by 10. If false, 'b' is increased by 10. Finally, 'b' is returned.

```
int a = 100;
float b = 50.75;
char c = 'x';
if(a>b) {
b = b - 10;
}
else {
b = b + 10;
}
return b;
```

Report text file:

```
File Edit View H1  B I 田 A5
Line 1: int -> Keyword
Line 1: a -> Identifier
Line 1: = -> Operator
Line 1: 100 -> Integer Constant
Line 1: ; -> Delimiter
Line 2: float -> Keyword
Line 2: b -> Identifier
Line 2: = -> Operator
Line 2: ; -> Delimiter
Line 3: char -> Keyword
Line 3: c -> Identifier
Line 3: = -> Operator
Line 3: ; -> Delimiter
Line 4: if -> Keyword
Line 4: ( -> Delimiter
Line 4: a -> Identifier
Line 4: > -> Operator
Line 4: b -> Identifier
Line 4: ) -> Delimiter
Line 4: { -> Delimiter
Line 5: b -> Identifier
Line 5: = -> Operator
Line 5: b -> Identifier
Line 5: - -> Operator
Line 5: 10 -> Integer Constant
Line 5: ; -> Delimiter
Line 6: } -> Delimiter
Line 7: else -> Keyword
Line 7: { -> Delimiter
Line 8: b -> Identifier
Line 8: = -> Operator
Line 8: b -> Identifier
Line 8: + -> Operator
Line 8: 10 -> Integer Constant
Line 8: ; -> Delimiter
Line 9: } -> Delimiter
Line 10: return -> Keyword
Line 10: b -> Identifier
Line 10: ; -> Delimiter
```

Summary:

Keywords = 6

Identifiers = 10

Integer Constants = 3

Operators = 8

Delimiters = 12

Symbole_table text:

```
File Edit View
```



```
1 a 1
2 b 2
3 c 3
```

Cmd output:

```
Done! tokens.txt and symbol_table.txt created successfully.
Total tokens founding number : 39
Total identifiers number: 3

Process returned 0 (0x0)   execution time : 0.083 s
Press any key to continue.
```

[illegible]